



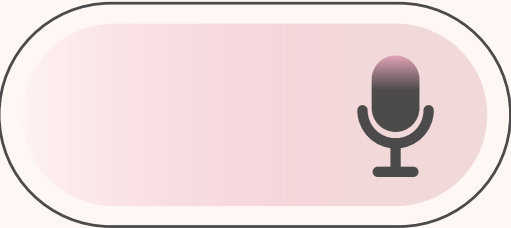
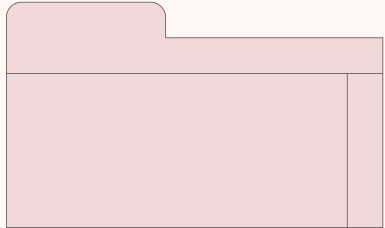
UNIVERSIDAD
DE ANTIOQUIA

B+TREE:

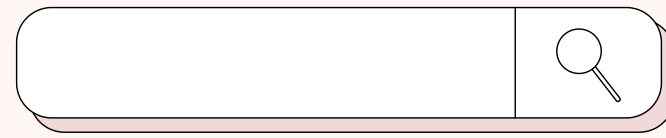
ESTRUCTURA Y PROPIEDADES

Por:

Juan Esteban Tobón Martínez
Laura Katherine Areiza Henao
Eyner Gomez Quintero
Sofia Espinal Vásquez

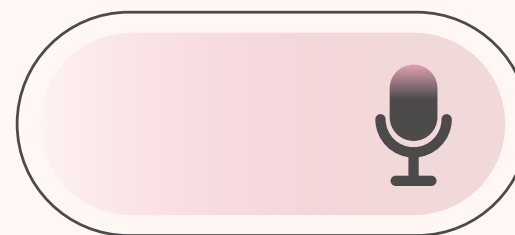
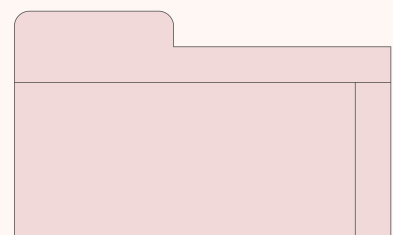
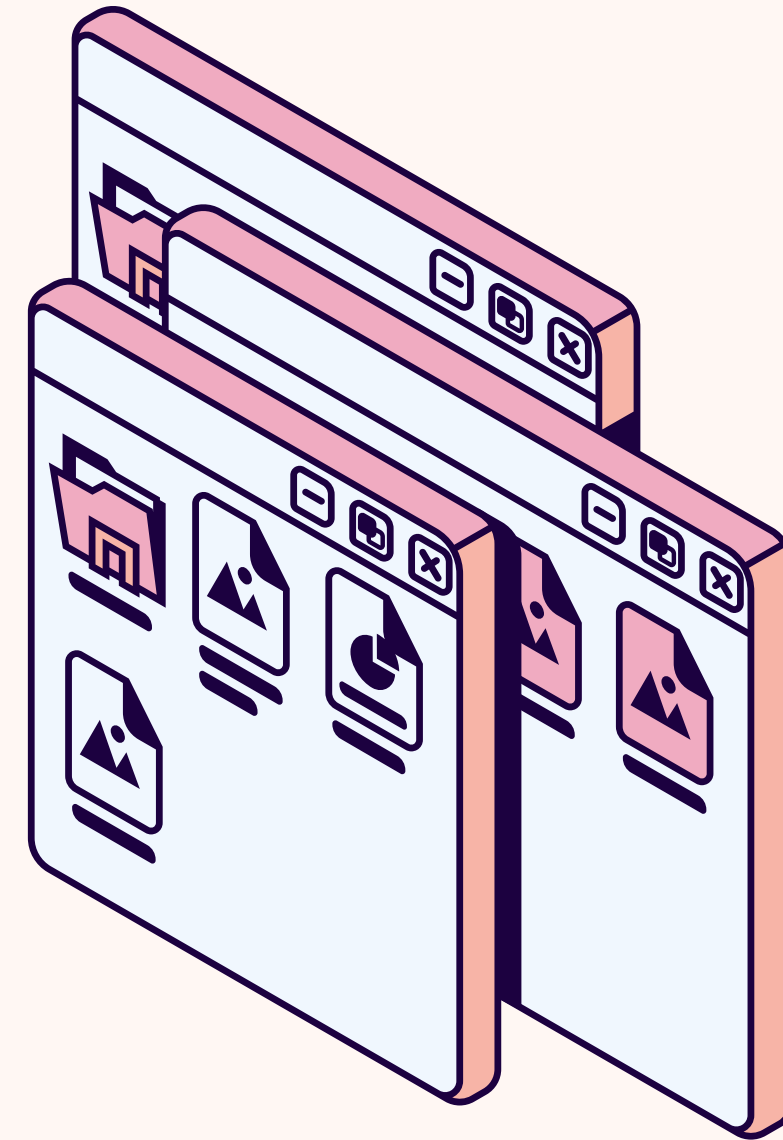


B+TREE

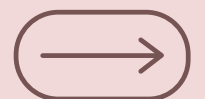


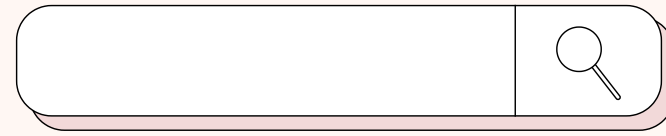
¿QUE ES UN B+TREE?

Una estructura de datos utilizada como índice en los sistemas de gestión de bases de datos. Que está diseñada para trabajar con almacenamiento en disco y permite minimizar los accesos a este y mejorar el rendimiento



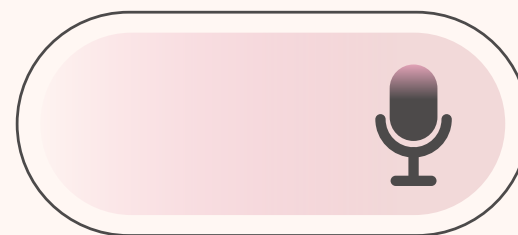
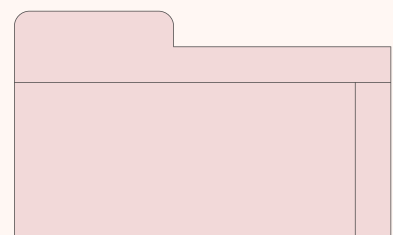
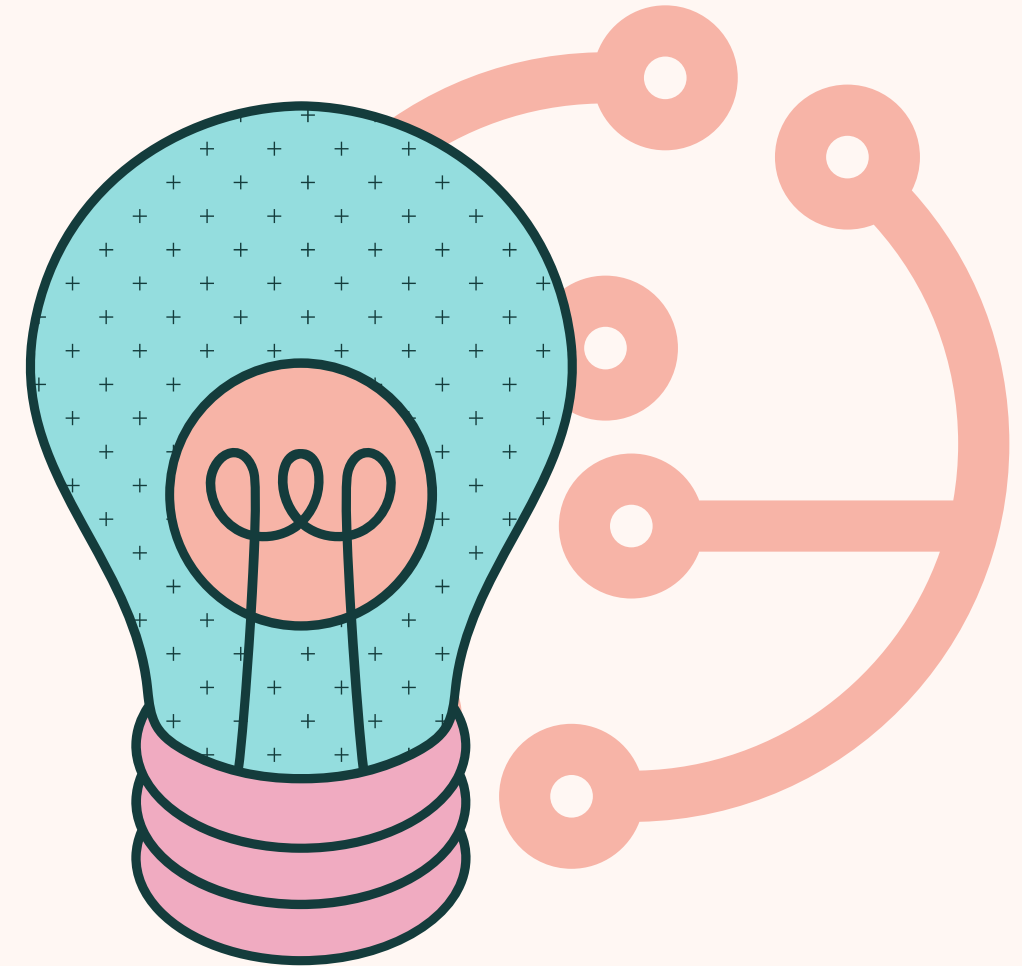
B+TREE



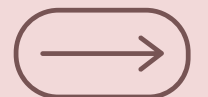


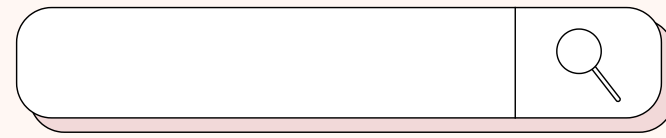
IMPORTANCIA

- Reduce la cantidad de accesos haciendo que se reduzca un poco el costo
- Mejora la eficiencia de las consultas



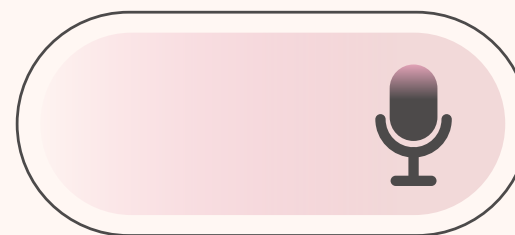
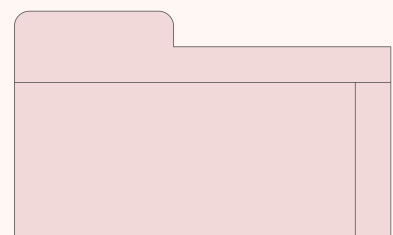
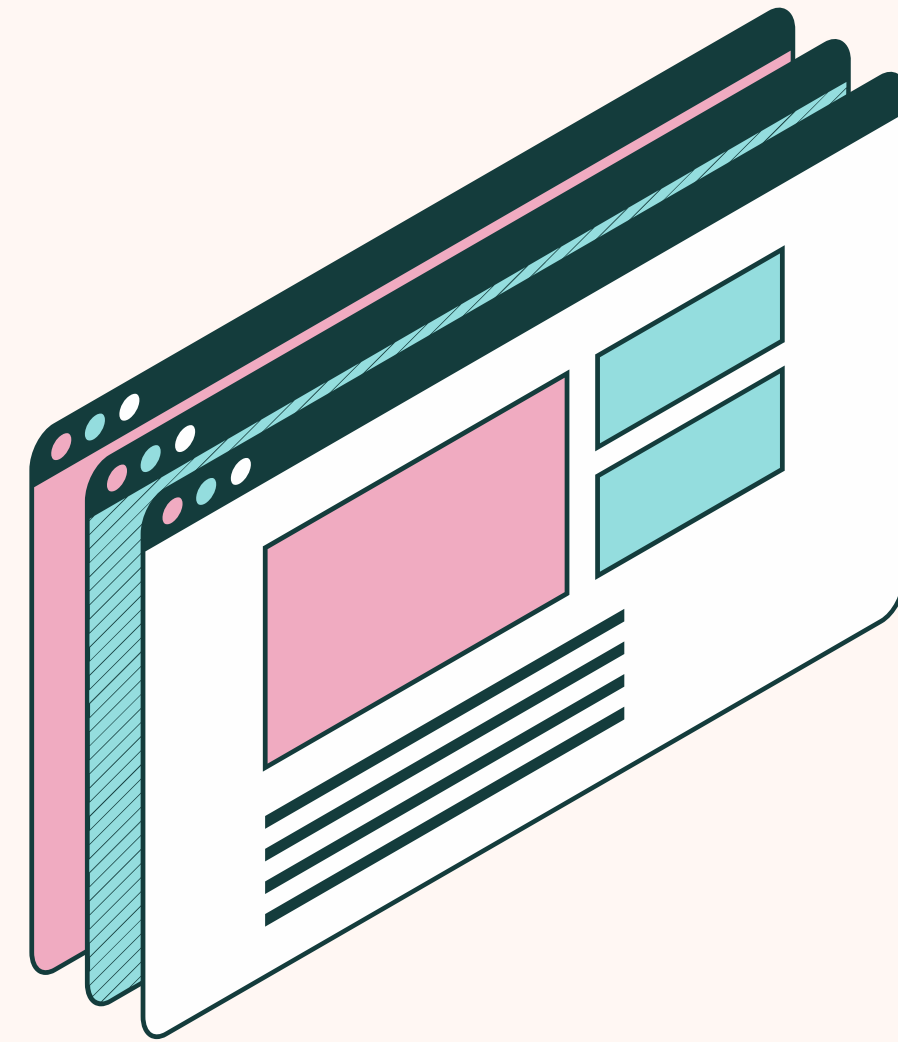
B+TREE



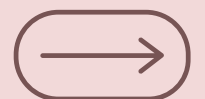


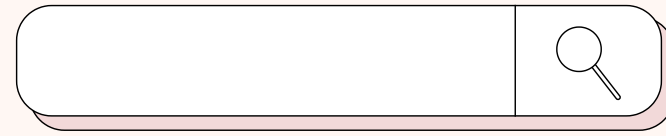
ORGANIZACION DEL B+TREE

- **Nodos internos:** contienen las claves que guían la búsqueda
- **Nodos hoja:** contienen los datos reales almacenados en disco.



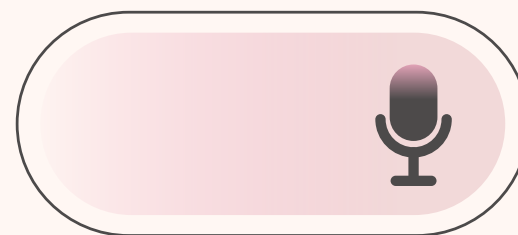
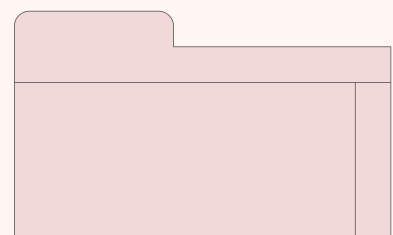
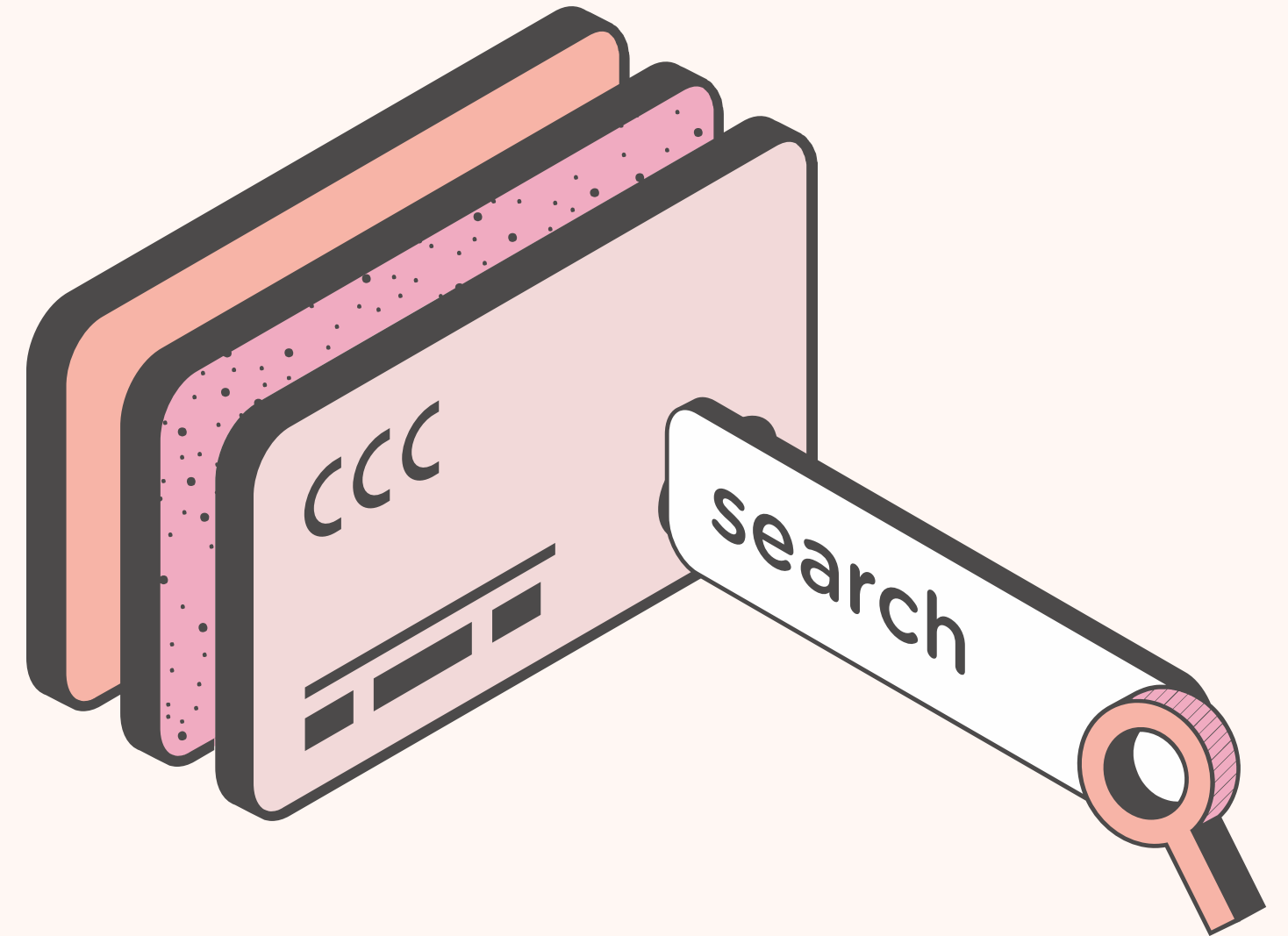
B+TREE

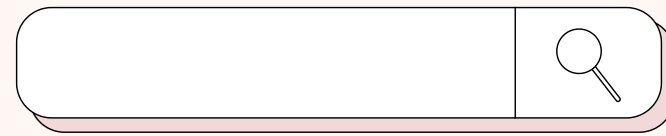




ENLACE ENTRE HOJAS

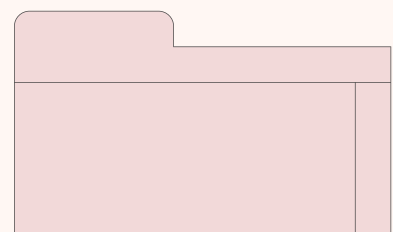
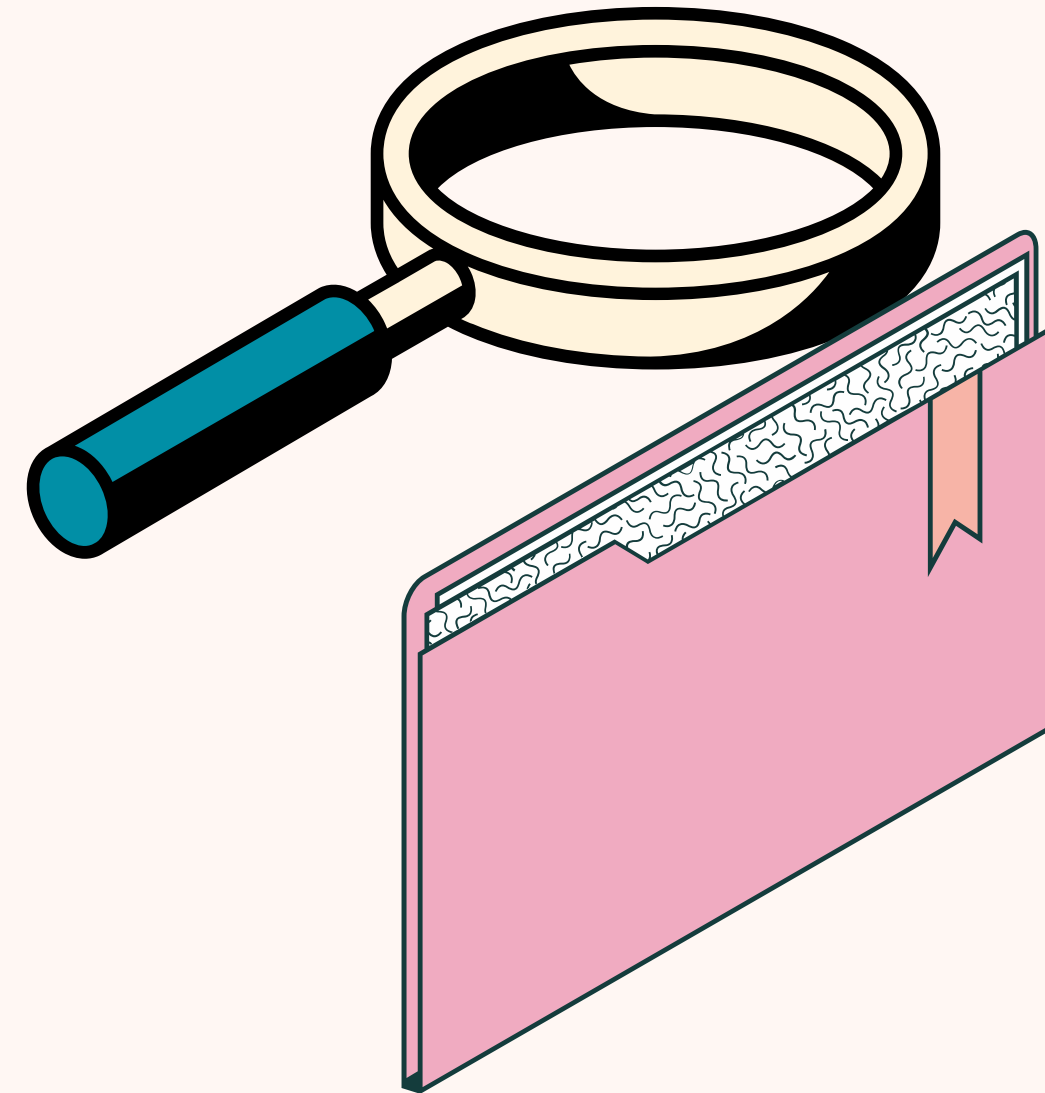
Una característica fundamental del B+Tree es que sus nodos hoja están enlazados entre sí, formando una estructura similar a una lista enlazada.



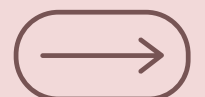


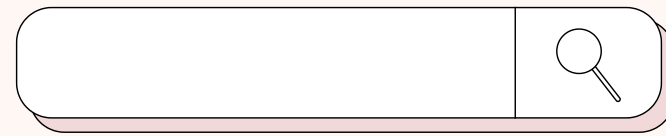
¿QUE ES LA BÚSQUEDA EN UN B+TREE?

Es el proceso que permite localizar un valor específico dentro del árbol. Consiste en recorrer la estructura desde la raíz hasta un nodo hoja, siguiendo un camino determinado por comparaciones entre el valor buscado y las claves de cada nodo.



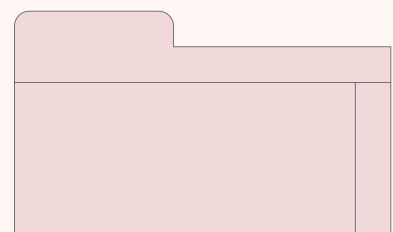
B+TREE



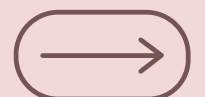


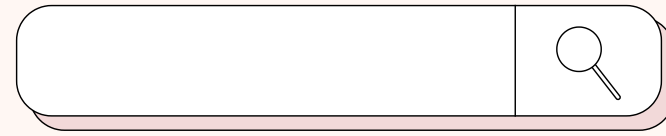
NAVEGACIÓN EN EL ÁRBOL

- Cada nodo divide los valores en intervalos
- Reglas:
 - Menor $\rightarrow$ primer puntero
 - Entre claves $\rightarrow$ puntero intermedio
 - Mayor o igual $\rightarrow$ último puntero
- Se repite el proceso hasta llegar a una hoja



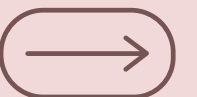
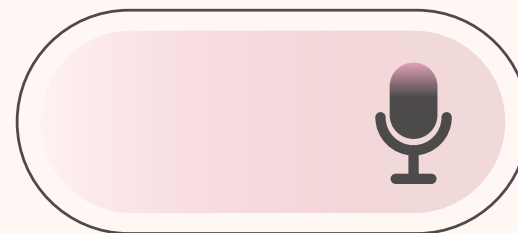
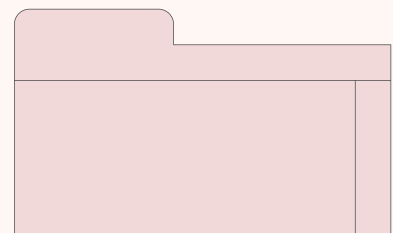
B+TREE





BÚSQUEDA EN NODO HOJA

- Cada nodo divide los valores en intervalos
- Reglas:
 - Menor $\rightarrow$ primer puntero
 - Entre claves $\rightarrow$ puntero intermedio
 - Mayor o igual $\rightarrow$ último puntero
- Se repite el proceso hasta llegar a una hoja



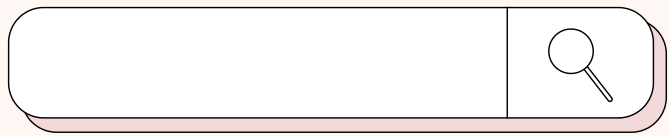


ILUSTRACIÓN (EXITOSA)

- El proceso inicia en la raíz:
- Se comparan las claves [20 | 40] con el valor 27
- 27 es mayor que 20 y menor que 40
- → Se sigue el puntero intermedio (subárbol central)
- Se desciende al nodo interno:

[25 | 30]

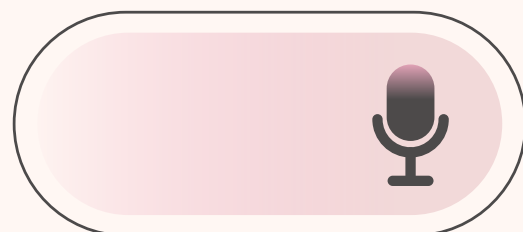
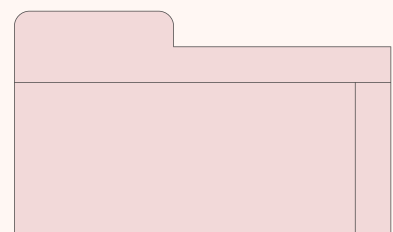
Y así se sigue la búsqueda...

En los nodos hoja, las claves se encuentran ordenadas y cada una está asociada a un registro o a un puntero hacia el registro correspondiente. Por lo tanto, el procedimiento consiste en verificar si el valor buscado coincide con alguna de las claves presentes en el nodo.

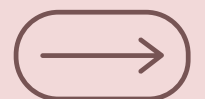
Nivel 0 (raíz)
[20 | 40]

Nivel 1 (nodos internos)
[10 | 15] [25 | 30] [45 | 50]

Nivel 2 (nodos hoja)
[1, 5, 9] [10, 12, 14] [15, 18, 19] ; [20, 22, 24] [25, 27, 29] [30, 35, 38] ; [40, 42, 44]
[45, 47, 49] [50, 55, 60]



B+TREE



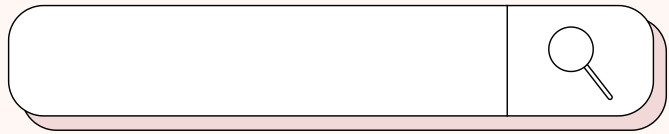
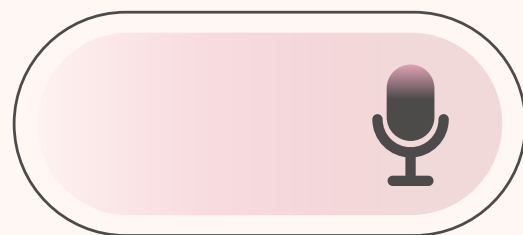
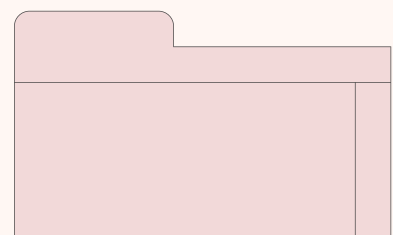
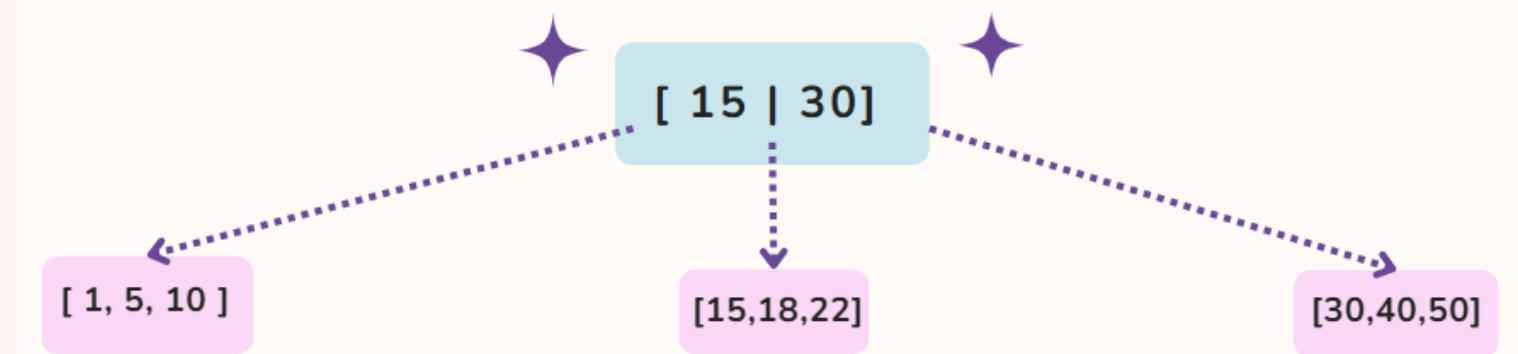
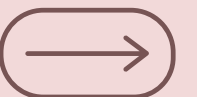


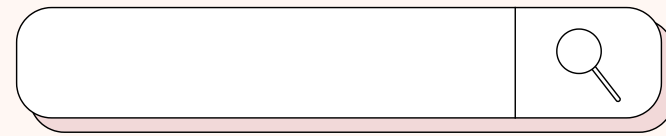
ILUSTRACIÓN (FALLIDA)

- El proceso inicia en el nodo raíz. Se compara el valor 19 con las claves:
 - 19 no es menor que 15
 - 19 no es mayor o igual que 30
- Por lo tanto, se encuentra en el intervalo entre 15 y 30
- Esto implica que se debe seguir el puntero P1, que corresponde al subárbol intermedio



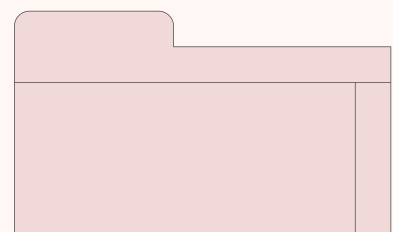
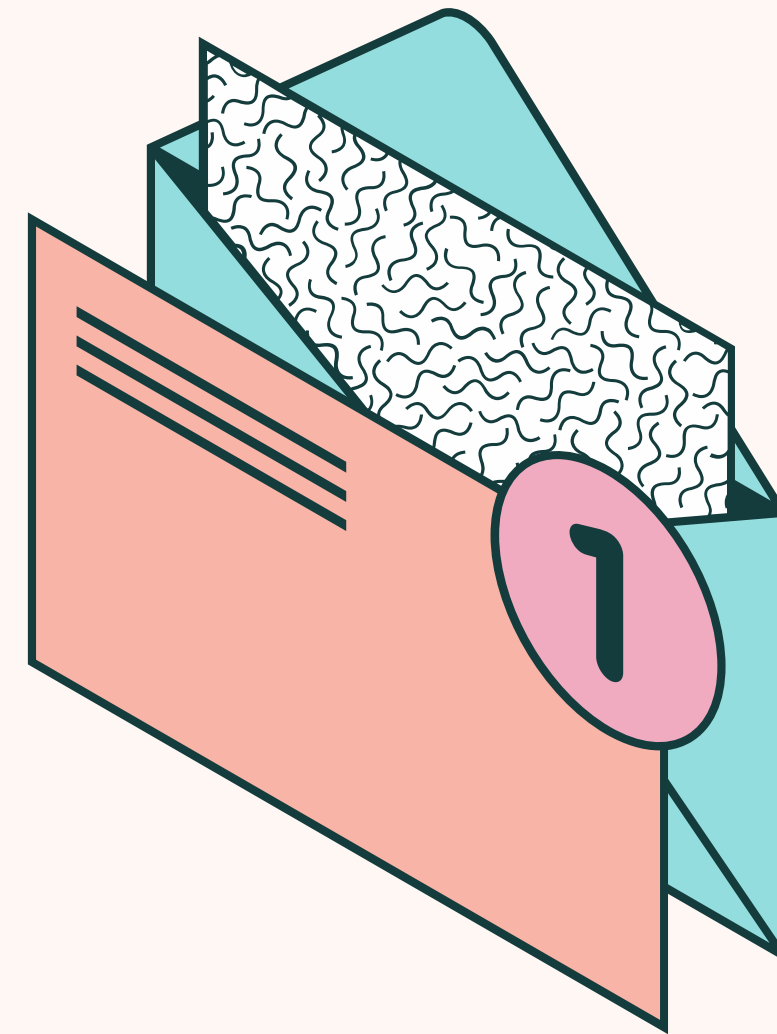
B+TREE



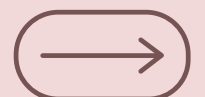


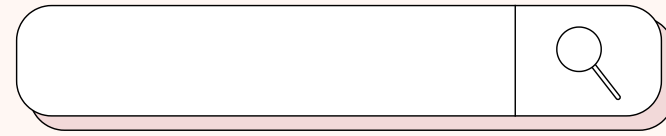
COMPLEJIDAD

Tiene una complejidad de $O(\log n)$, lo que significa que el número de pasos crece de forma logarítmica y se mantiene eficiente incluso con grandes volúmenes de datos.



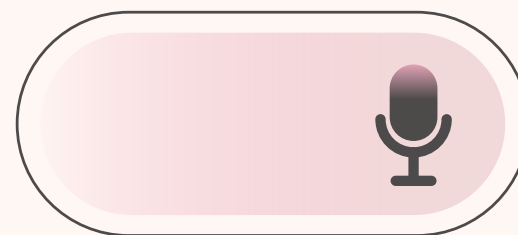
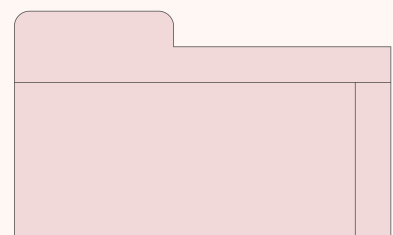
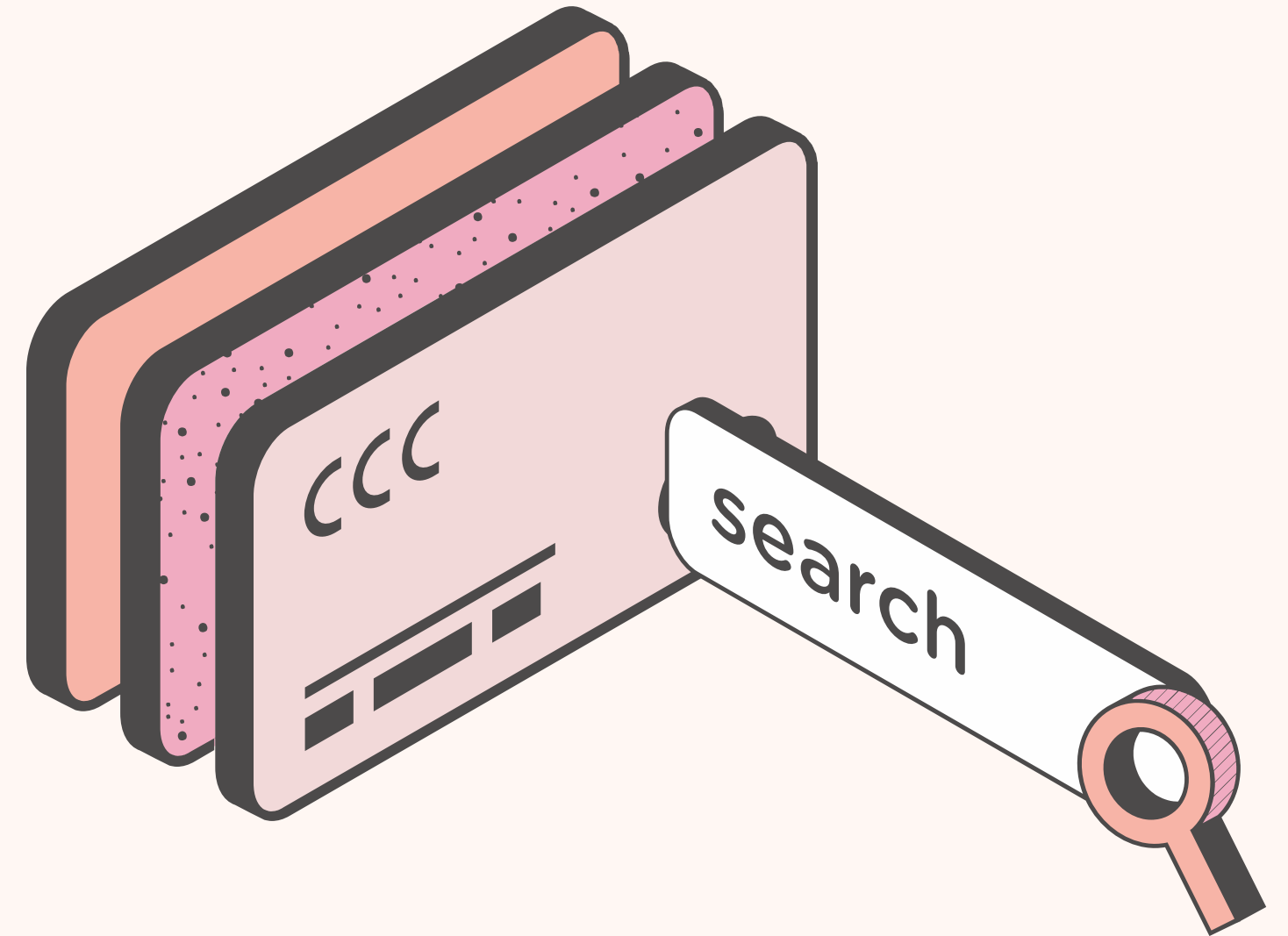
B+TREE



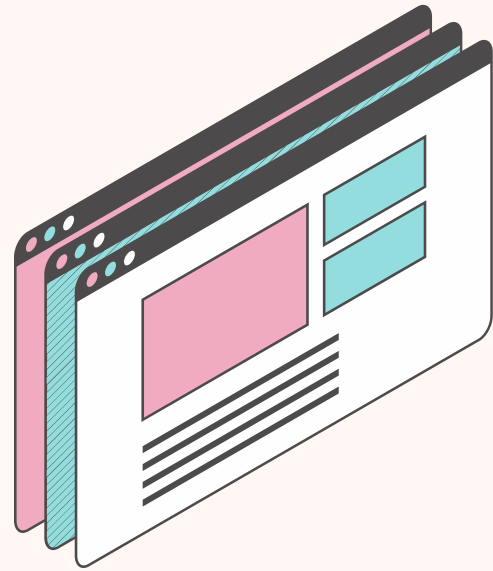


¿QUE ES UN RANGE QUERY?

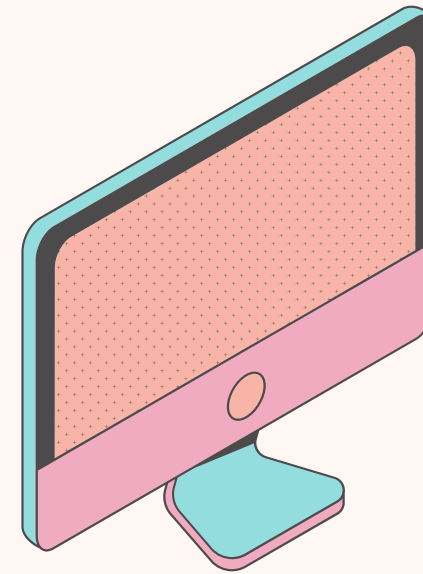
Un range query es una operación que permite recuperar todos los registros cuyos valores se encuentran dentro de un intervalo determinado. En lugar de buscar un solo valor busca un conjunto de valores dentro de un rango



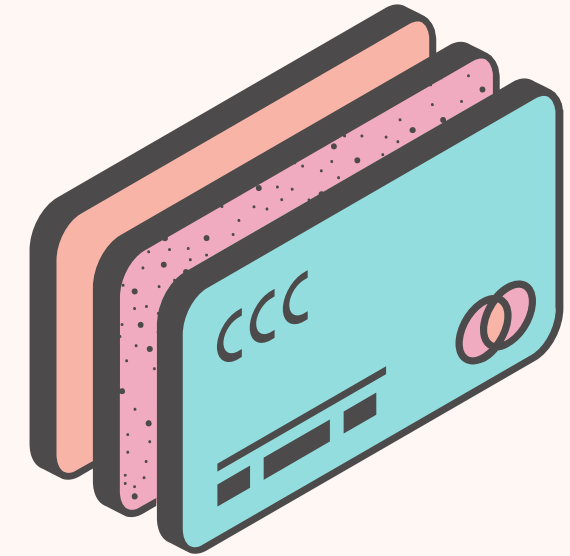
¿POR QUÉ ES IMPORTANTE?



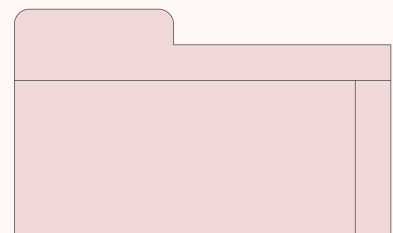
Todas las claves se
almacenan en los
nodos hoja



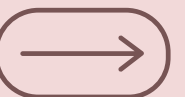
Las hojas están conectadas
entre sí mediante punteros
(lista enlazada)

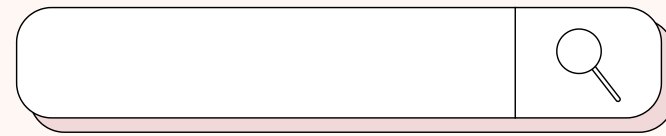


Los datos están ordenados



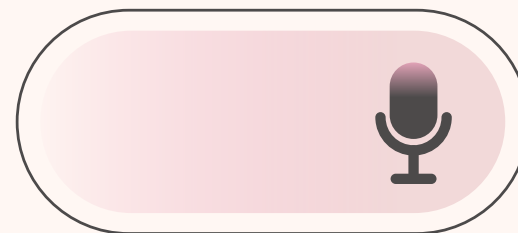
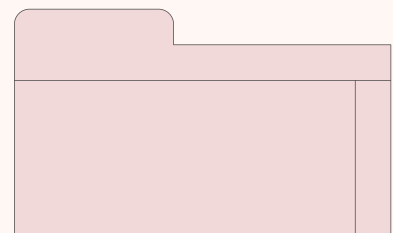
WWW.REALLYGREATSITE.COM





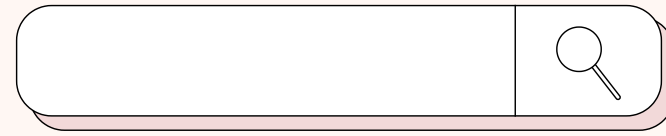
EJEMPLO DE IMPLEMENTACIÓN

```
RangeQuery(T, a, b):  
    nodo = buscarHoja(T, a)  
  
    mientras nodo != NULL:  
        para cada clave k en nodo:  
            si k >= a y k <= b:  
                imprimir k  
            si k > b:  
                terminar  
  
    nodo = nodo.siguiente
```



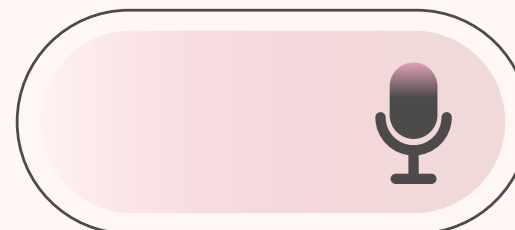
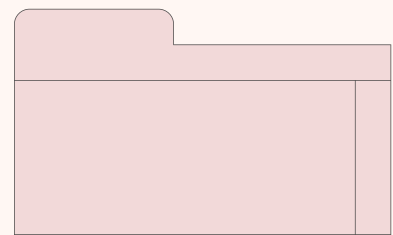
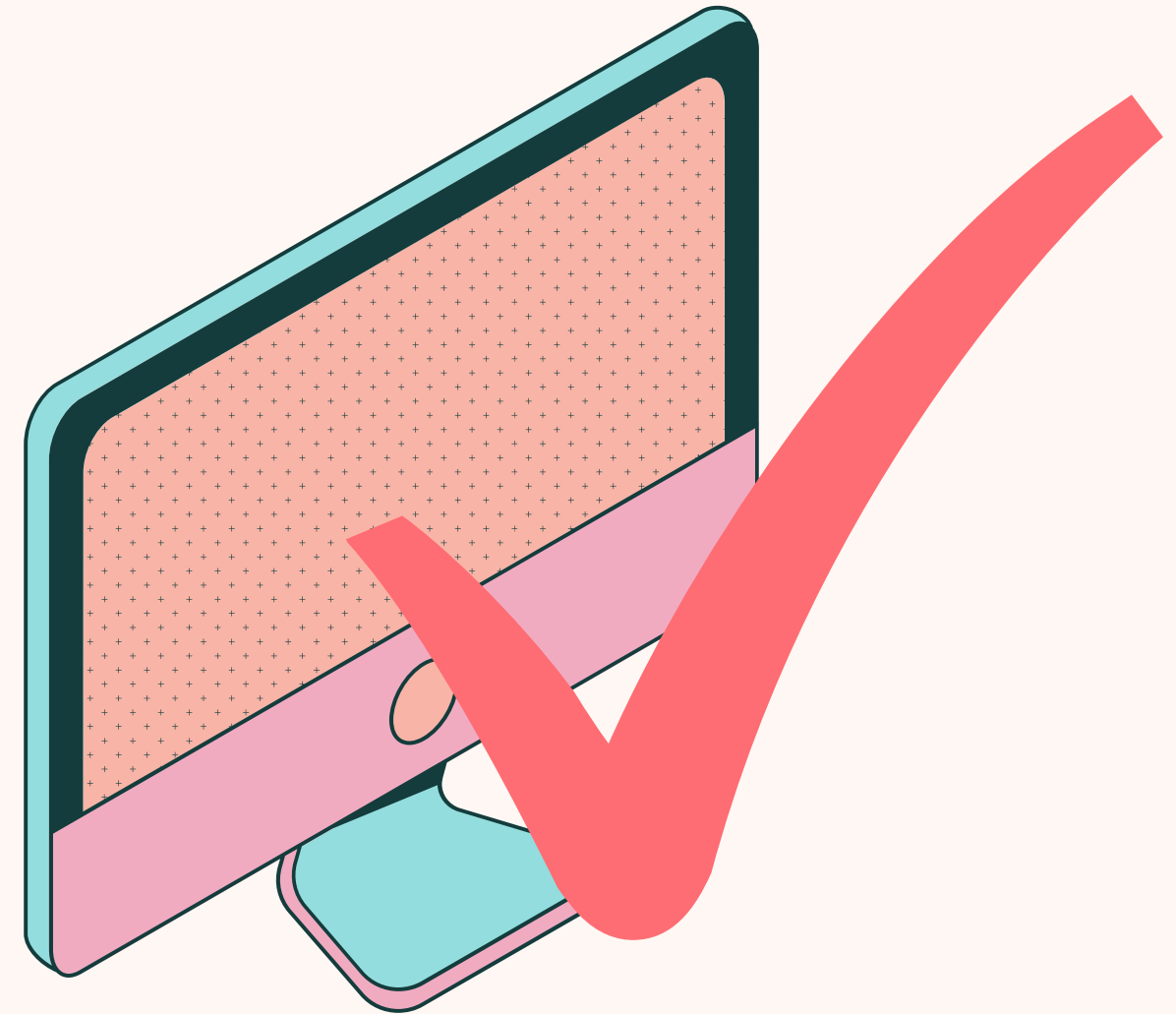
B+TREE





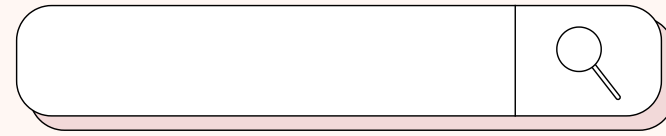
VENTAJAS

- Acceso ordenado
- Recorrido secuencial eficiente
- Alto rendimiento en disco
- Ideal para bases de datos



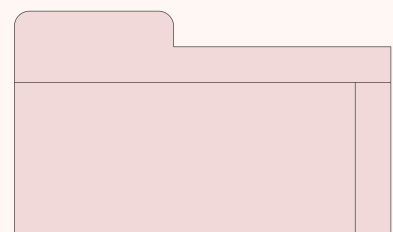
B+TREE





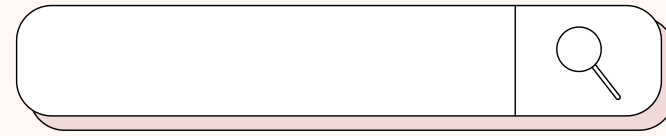
EFICIENCIA DEL B+TREE EN BUSQUEDA DE DATOS

En términos de complejidad, esta estructura tiene una complejidad de $O(\log n)$, es decir, se realiza en tiempo logarítmico. Esto permite que, incluso si la cantidad de datos aumenta considerablemente, el tiempo de búsqueda crezca de forma controlada y eficiente.



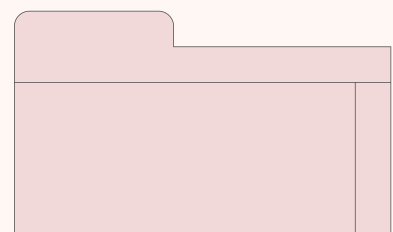
B+TREE

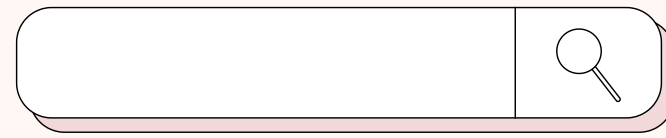




EFICIENCIA EN CONSULTAS POR RANGO

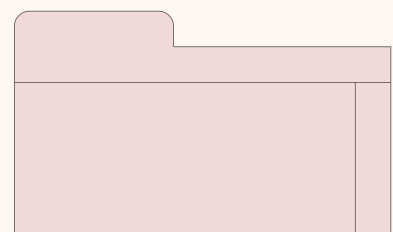
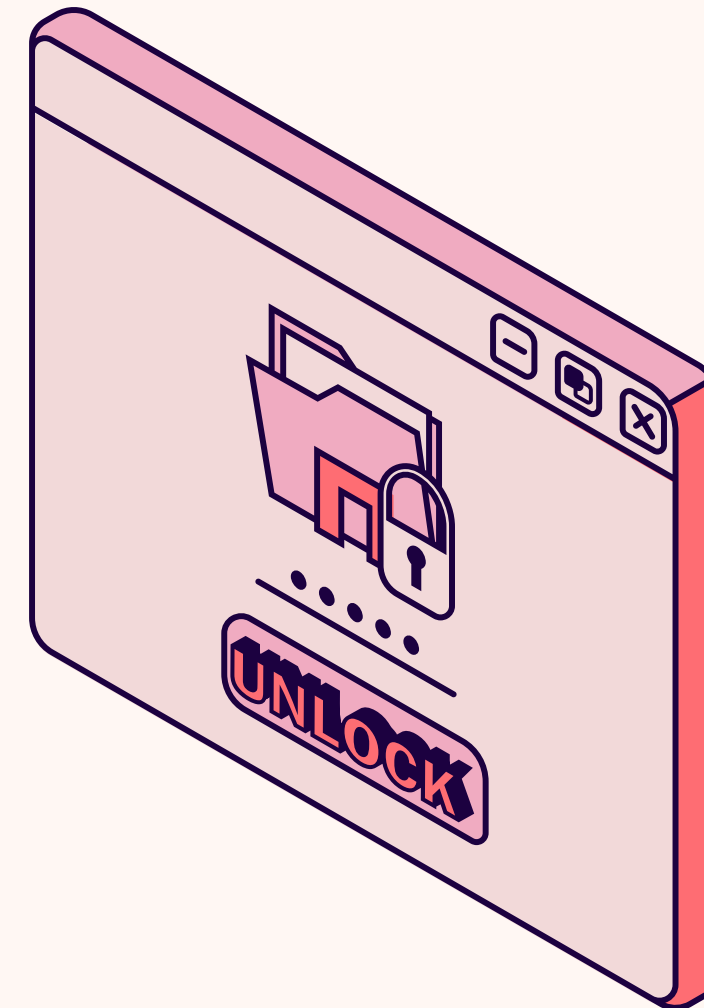
Una de las principales ventajas de esta estructura es su capacidad de manejar consultas por rango de forma eficiente. Este tipo de consultas consiste en recuperar los valores comprendidos dentro de un rango, por ejemplo, entre 20 y 50.



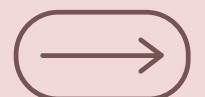


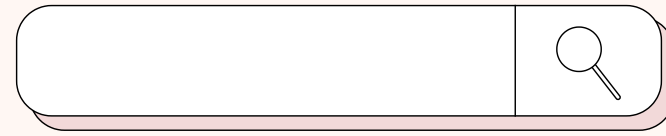
OPERACIONES ENTRADA / SALIDA

El diseño de esta estructura está orientado a minimizar el número de accesos al disco, ya que estas son una de las operaciones más costosas en los sistemas de bases de datos. Debido a que cada nodo puede almacenar múltiples claves, se reduce la altura del árbol y, en consecuencia, el número de accesos necesarios para realizar una búsqueda.



B+TREE





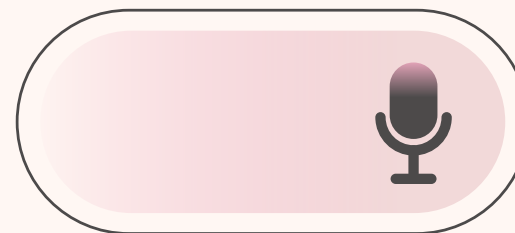
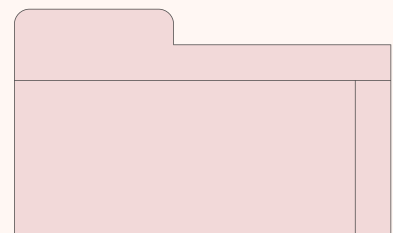
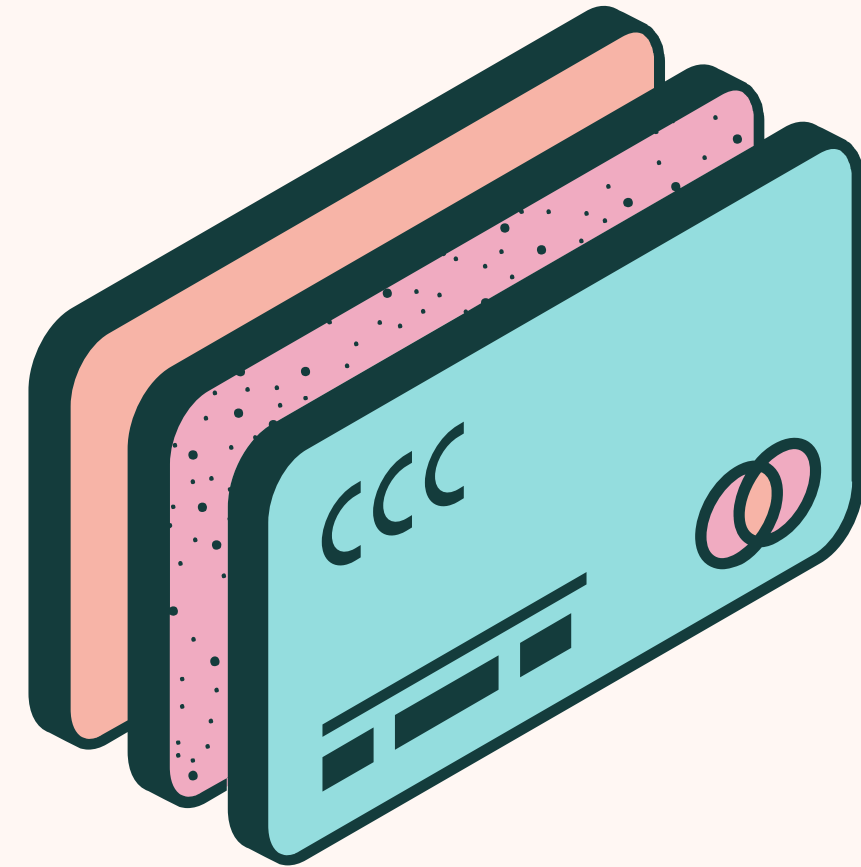
COMPARACIÓN B+TREE / B-TREE

B-Tree:

- Datos en todo el árbol
- Menos eficiente para recorridos

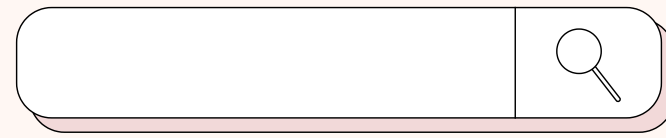
B+Tree:

- Datos en hojas
- Mejor para consultas por rango
- Permite recorrido secuencial



B+TREE

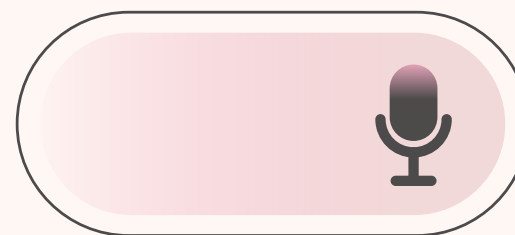
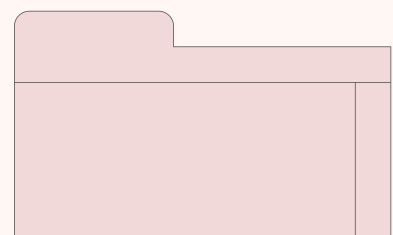
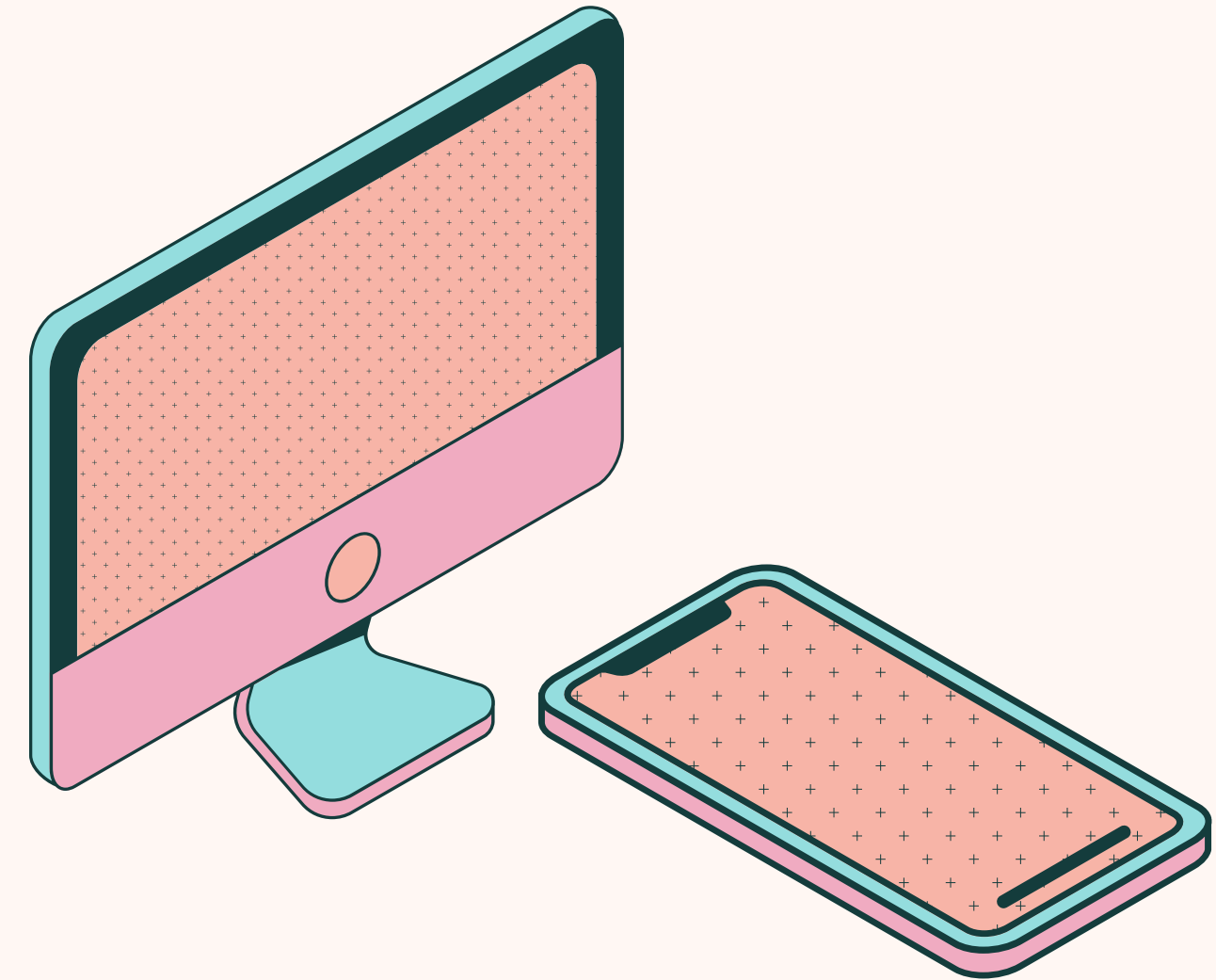




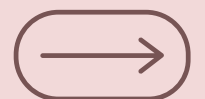
APLICACIONES DEL B+TREE

Esta estructura se utiliza principalmente en sistemas donde se requiere acceso rápido a grandes cantidades de datos. Sus principales aplicaciones incluyen:

- Índices en bases de datos relacionales
- Sistemas de archivos
- Motores de almacenamiento de bases de datos



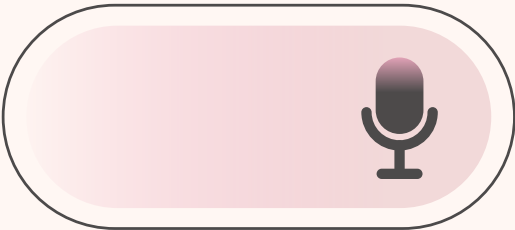
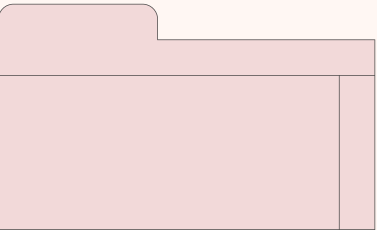
B+TREE





UNIVERSIDAD
DE ANTIOQUIA

THANK
YOU!



B+TREE